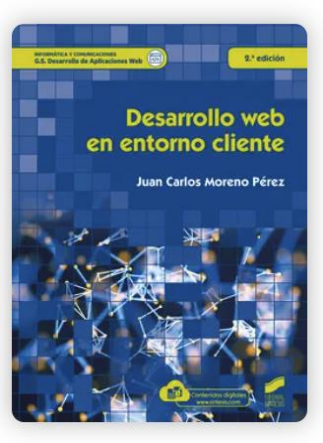
Desarrollo web en entorno cliente

Unidad 1
Selección de arquitecturas y herramientas de programación

2º DAW · Curso 26-27

Libro recomendado

Material de referencia en el que se apoya este módulo



Desarrollo web en entorno cliente (2.ª ed.)
Juan Carlos Moreno Pérez · Editorial Síntesis
ISBN 978-84-1357-709-8

Por qué este libro

Sigue el temario del módulo de Desarrollo Web en Entorno Cliente y ordena los contenidos igual que la unidad.

Qué aporta frente a las diapositivas

Explicaciones más extensas, ejemplos de código completos y actividades resueltas para practicar.

Cómo usarlo

Estas diapositivas son un resumen de apoyo en clase; el libro es la referencia para ampliar y estudiar.

Aviso: esta presentación es material docente de elaboración propia y no sustituye a la obra original.
No existe relación comercial ni patrocinio con la editorial; la recomendación es únicamente académica y los derechos pertenecen a sus autores.

Contenidos de la unidad

Unidad 1 · Selección de arquitecturas y herramientas de programación

1.1 Desarrollo de software

Proceso de creación y metodologías.

- Cascada, Agile y MVP
- 1.1.1 Tipos de aplicaciones
- 1.1.2 Aplicaciones web

1.2 Lado cliente y lado servidor

Front-end, back-end y tecnologías.

- 1.2.1 Tres niveles
- 1.2.2 Frameworks y MVC
- 1.2.3 ¿Por qué JavaScript?

1.3 Herramientas del lado cliente

El kit de trabajo para programar.

- 1.3.1 Navegador y DevTools
- 1.3.2 Editor de código
- 1.3.3 Intérprete de JS
- 1.3.4 Git y VS Code

1.4 Características de los lenguajes de script

Qué son y qué los caracteriza.

- Interpretados y embebidos
- Tipado dinámico
- Multiplataforma

1.5 Integración de JavaScript dentro de HTML

Cómo se integra en la página.

- Externo, embebido e inline
- Dos ejemplos comparados

1.7 Posibilidades que ofrece JavaScript

Tres ejemplos de lo que permite.

- 1.7.1 Cambiar contenido
- 1.7.2 Cambiar atributos
- 1.7.3 Cambiar estilo CSS

1.8 Comunicación de JavaScript con el exterior

Cuatro formas de dar mensajes.

- console.log e innerHTML
- document.write y alert

De la teoría del desarrollo a tus primeros programas en JavaScript.

Los apartados 1.4 a 1.8 llevan la unidad del concepto a la práctica con JavaScript.

Introducción

Por qué esta unidad

01

Internet lo cambió todo

La informática experimentó un desarrollo exponencial con la llegada de internet.

02

Hacen falta profesionales

Los sitios web necesitan quien los desarrolle e infraestructuras que los soporten.

03

Un campo muy amplio

El desarrollo web abarca más tecnologías de las que cabe estudiar en una unidad.

> Por eso esta unidad se centra en el desarrollo del lado cliente: lo que se ejecuta en el navegador.

1.1 Desarrollo de software

Proceso de creación de aplicaciones en cascada (waterfall)

1. Planificación y análisis

Jefe de proyecto y cliente definen el alcance.

2. Definición de requisitos

De forma clara y concisa, validada por el cliente.

3. Diseño de la arquitectura

Módulos, flujos de datos e integraciones.

4. Construcción

El lenguaje depende de los estándares y del tipo de software.

5. Pruebas

Unitarias, de integración, de estrés…

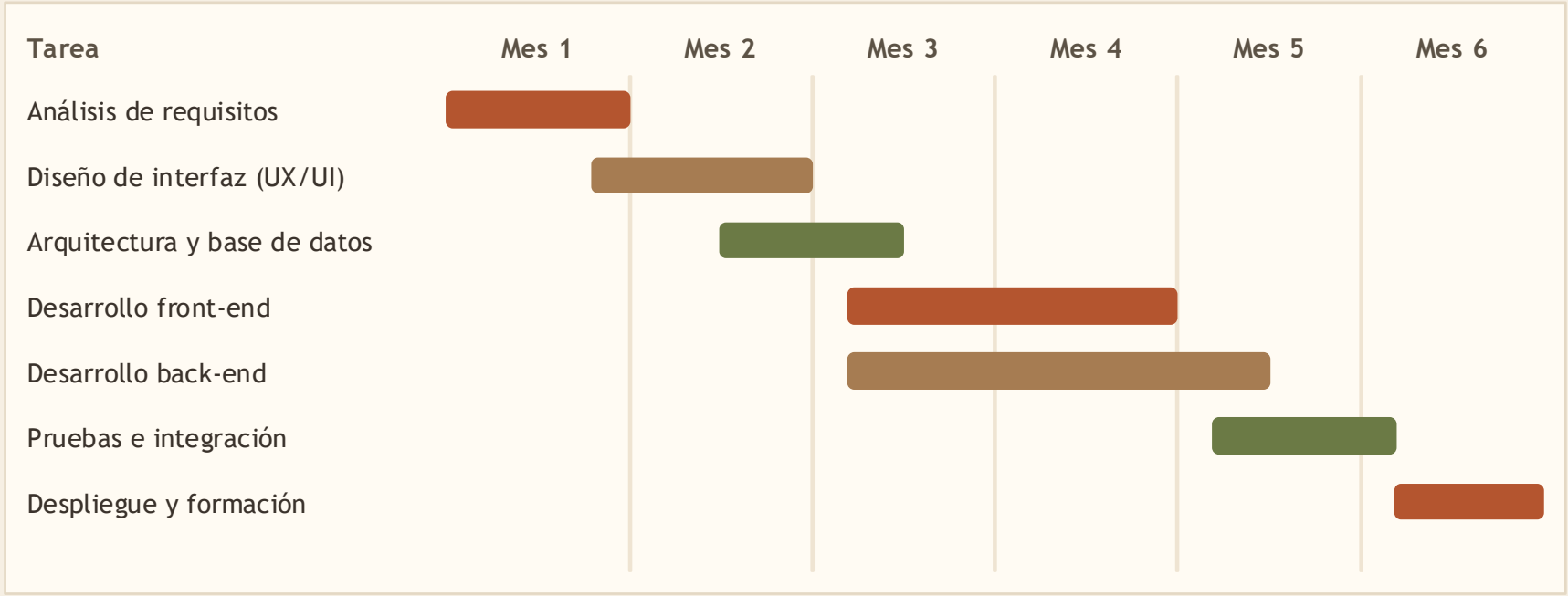
6. Despliegue y mantenimiento

Paso a producción; conviene volver a probar.

En la planificación se asigna un porcentaje de esfuerzo a cada fase.

1.1 Planificación del proyecto

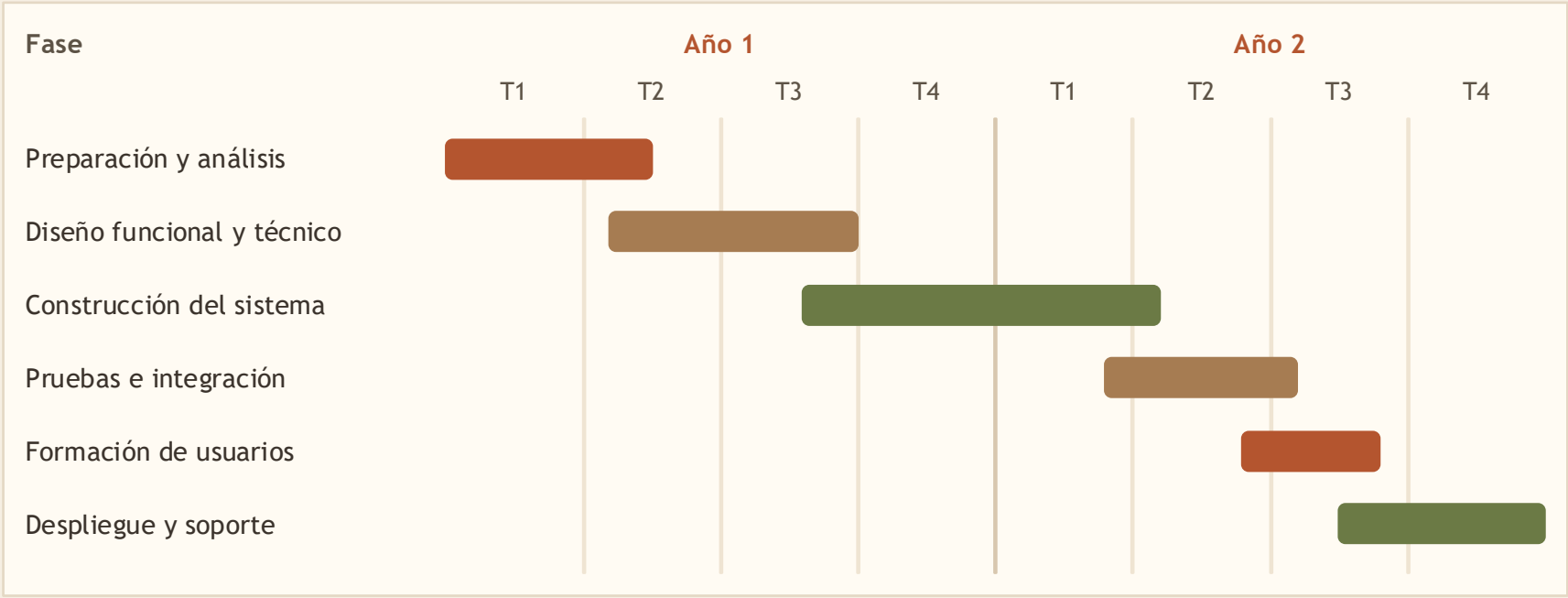
Ejemplo de cronograma (Gantt) de un proyecto web de seis meses



Cada barra muestra cuándo empieza y termina una tarea; el solape entre front-end y back-end indica trabajo en paralelo.

1.1 Planificación por fases

Ejemplo de proyecto plurianual: las fases se solapan a lo largo de dos años



Las fases no son estancas: el diseño arranca antes de cerrar el análisis y la formación se solapa con las pruebas.

1.1 Seguimiento del desarrollo

Ejemplo de cronograma con fechas, duración y responsable de cada tarea

Tarea	Inicio	Fin	Días	Responsable	Mayo — Julio
Análisis funcional	04/05	15/05	10	Ana G. · Analista	
Diseño de interfaz	11/05	29/05	15	Luis M. · UX	
Desarrollo de módulos	25/05	26/06	25	Equipo dev	
Carga de datos	15/06	30/06	12	Marta R. · DBA	
Formación de usuarios	23/06	04/07	10	Ana G. · Analista	
Pruebas de aceptación	01/07	15/07	11	Cliente + QA	
Implantación	16/07	22/07	5	Equipo dev	

Un cronograma de proyecto asigna a cada tarea fechas de inicio y fin, duración en días y persona responsable.

1.1 Coste del desarrollo

Tarifas por perfil profesional

CATEGORÍA	
CONSULTOR A	491,07 €
CONSULTOR B	474,51 €
GESTOR DE PROYECTO TEC.A	462,22 €
GESTOR DE PROYECTO TEC.B	424,71 €
ANALISTA A	408,98 €
ANALISTA B	373,89 €
ANALISTA C	356,35 €
ANALISTA D	338,80 €
PROGRAMADOR A	307,82 €
PROGRAMADOR B	279,28 €
PROGRAMADOR C	252,29 €
PROGRAMADOR D	216,59 €

El coste de un proyecto depende del perfil asignado: un consultor casi duplica la tarifa diaria de un programador.

¿Qué es la metodología Agile?

Entregas pequeñas y frecuentes en lugar de un único gran entregable final

1

Backlog

Lista priorizada de lo que aporta valor

2

Sprint

Se elige un bloque para 2-4 semanas

3

Desarrollo

El equipo construye y prueba a diario

4

Entrega

Software funcionando al final del sprint

5

Revisión

El cliente valida y se ajusta el plan

El ciclo se repite sprint tras sprint: cada vuelta añade funcionalidad y recoge la opinión del cliente.

Iteraciones cortas
Valor entregado cada pocas semanas

Cliente implicado
Valida y prioriza de forma continua

Cambio bienvenido
Los requisitos pueden evolucionar

Agile parte de un MVP.

Producto Mínimo Viable: la versión más pequeña que ya aporta valor real

MVP
Versión mínima que ya resuelve el problema principal

v1
Se añade lo que el usuario pide tras usarlo

v2
Nuevas funciones y mejoras de rendimiento

Producto
Solución completa, madurada con cada iteración

Cada entrega parte de lo anterior: se construye, se mide el uso real y se aprende para decidir el siguiente paso.

Mínimo, no incompleto
Poco alcance, pero utilizable de principio a fin

Se lanza pronto
Antes de invertir meses en funciones sin validar

Evoluciona con datos
El feedback marca qué se añade en cada iteración

Agile frente a Waterfall

Dos formas de organizar un proyecto de desarrollo

Waterfall · secuencial

Las fases se ejecutan una detrás de otra y no se vuelve atrás: primero se cierran los requisitos, luego el diseño, la construcción, las pruebas y el despliegue. El cliente ve el resultado al final. Funciona bien cuando el alcance está claro y no va a cambiar, pero cualquier corrección tardía resulta muy cara.

Agile · iterativo

El trabajo se divide en iteraciones cortas (sprints) que entregan software funcionando cada pocas semanas. El cliente valida de forma continua y los requisitos evolucionan. Encaja en proyectos con incertidumbre, aunque exige una buena gestión del backlog.

1.1.1 Tipos de aplicaciones

Ya no se clasifican solo por dispositivo: la misma app se usa en varios

- Personalizadas: cuando el software comercial no cubre requisitos específicos.
- Desarrollo rápido: incremental, con varios equipos por módulos.
- Low-code / no-code: interfaces visuales (Appian, ServiceNow, Power Apps).
- Móviles: para cualquier sistema operativo móvil.
- Bases de datos: recopilan información para analizarla.
- Empresariales: ERP como SAP o Dynamics 365 Business Central.
- Web: alojadas en un servidor y accesibles desde cualquier dispositivo.

Enfoques actuales

Cómo se decide hoy el punto de partida de un proyecto

Mobile first

Mejora progresiva: se prioriza el diseño y desarrollo para móvil por encima del escritorio.

Cloud first

Se opta por soluciones en la nube antes que locales, suscribiéndose a un proveedor en vez de construir el stack propio.

1.1.2 Desarrollo de aplicaciones web

Construcción y mantenimiento de sitios web

Queremos que se vea bien, funcione rápido y ofrezca una buena experiencia de usuario (UX).

La elección del lenguaje de programación depende de:

Las tareas

Qué debe resolver realmente la aplicación.

La organización

Sus normas y su plataforma tecnológica.

El equipo

El conocimiento de los desarrolladores.

1.2 Lado cliente y lado servidor

Front-end · lado cliente

Aquello con lo que interactúa el usuario y se ejecuta en el navegador. Se desarrolla el diseño de la interfaz y su experiencia (UX).

Back-end · lado servidor

Analiza las acciones del usuario y devuelve el resultado. Contiene la lógica de negocio y almacena los datos.

Los perfiles full stack desarrollan tanto en front como en back, normalmente con JavaScript.

1.2 Front-end y back-end

Una analogía visual



> Como en un bordado, el front-end es la cara pulida que ve el usuario y el back-end la trama que lo sostiene.

1.2 Front-end y back-end en la arquitectura de tres capas

Qué hace cada lado y en qué nivel se ejecuta

FRONT-END
lado cliente

Presentación · Navegador

Interfaz que ve y usa el cliente: HTML, CSS y JavaScript. Valida formularios y envía peticiones al servidor.

petición HTTP / API

BACK-END
lado servidor

Aplicación · Servidor

Lógica de negocio y APIs: PHP, Java, Python o Node.js. Recibe la petición, la procesa y devuelve la respuesta.

consulta a los datos

Datos · Base de datos

Almacena y recupera la información: MySQL, PostgreSQL o MongoDB. Solo accesible desde el nivel de aplicación.

El usuario solo ve el front-end; el back-end ejecuta la lógica y guarda los datos, sin exponerse al navegador.

1.2.1 Arquitectura de tres niveles

Cada nivel se ejecuta en su propia infraestructura y se desarrolla en paralelo

Presentación

Interfaz de usuario: muestra y recopila información. Se ejecuta en el navegador con HTML, CSS y JavaScript.

Aplicación

Servidor de aplicaciones con la lógica de negocio. Python, Java, PHP o Ruby; se comunica con los datos vía API.

Datos

Servidor de base de datos donde se almacena la información: relacional o NoSQL (MongoDB, Cassandra).

La comunicación siempre pasa por el nivel de aplicación: presentación y datos no se comunican entre sí.

1.2.1 Arquitectura de tres niveles

Diferencias entre cliente y servidor

Característica	Lado cliente	Lado servidor
Definición	Componentes visuales de la aplicación web	Funcionalidades, acceso a datos y administración de servidores
Habilidades	HTML, CSS, SASS, JavaScript	Python, Ruby, Java, PHP
Independencia	No funciona sola, salvo sitios estáticos	Funciona de forma independiente del front-end
Objetivo	Acceso y respuesta en todos los dispositivos	Escalabilidad, baja latencia y ejecución sin fallos
Frameworks	AngularJS, React, Vue.js	Django, Flask, CakePHP, Laravel, Ruby on Rails
Habilidades extra	Comprensión de diseño de UI y UX	Razonamiento lógico y resolución de problemas

1.2.2 Otras tecnologías, frameworks y stack tecnológico

Qué aporta cada tecnología y cómo se reparte el stack entre cliente y servidor

SASS

Preprocesador de CSS con variables y funciones. Reutiliza código, pero debe compilarse.

jQuery

Librería de JavaScript para HTML, eventos y animaciones. Reduce líneas de código.

Vue.js

Framework creado por Evan You para aplicaciones web compactas y flexibles.

AngularJS

Framework de Google con arquitectura MVC, orientado a aplicaciones de una sola página.

React

Framework de Facebook centrado en componentes reutilizables de interfaz de usuario.

La elección

Depende de la aplicación, de la empresa y del conocimiento del equipo.

Front-end · lado cliente

Se ejecuta en el navegador

Lenguajes: HTML5 · CSS3 · JavaScript

Frameworks: Angular · React · Vue.js · Ionic

Estilos y herramientas: SASS · Bootstrap · webpack · npm

Back-end · lado servidor

Se ejecuta en el servidor

Lenguajes: Java · Python · PHP · Ruby · C#

Frameworks: Spring · Java EE/JSF · .NET · Django

Entorno y datos: Node.js · npm · JUnit · MySQL · MongoDB

JavaScript es el único lenguaje presente en los dos lados: en el navegador y, con Node.js, en el servidor.

1.2.2 Tecnologías del lado cliente

El marco de ejecución de las aplicaciones web es el navegador

HTML

Lenguaje de marcas que define la estructura de la página. Simple y compatible, pero estático. Permite asociar eventos a los elementos.

CSS

Define el estilo visual de las páginas. Difícil para principiantes y con incompatibilidades entre navegadores. Modificable desde JavaScript.

JavaScript

Lenguaje principal de la lógica de cliente: interacciones dinámicas, manejo de eventos y validación antes de enviar al servidor.

1.2.2 Qué es un framework de cliente y qué es el MVC

Una base de trabajo ya hecha y un patrón para organizar el código

Framework de cliente

Conjunto de librerías, normas y estructura ya preparadas que se ejecutan en el navegador y evitan programar todo desde cero.

- **Estructura lista**
  Carpetas, rutas y convenciones ya definidas.
- **Componentes reutilizables**
  Piezas de interfaz que se reutilizan.
- **Menos código repetido**
  El framework resuelve eventos, datos y vistas.

Vista

Lo que el usuario ve: la interfaz, en HTML y CSS. Muestra los datos del modelo.

el usuario actúa sobre la interfaz

Controlador

Recibe las acciones del usuario, decide qué hacer y conecta vista y modelo.

el controlador pide o actualiza los datos

Modelo

Los datos y las reglas de negocio: los pide al servidor y los mantiene actualizados.

En MVC el modelo devuelve los datos a la vista, así la interfaz se actualiza sin mezclar diseño y lógica; Angular sigue este patrón.

1.2.3 ¿Por qué JavaScript?

El lenguaje de referencia del lado cliente

95 %

de las aplicaciones web se desarrollan en JavaScript

1. Es muy fácil de implementar: basta con incluir el código en el documento HTML.
2. Funciona en todos los navegadores, incluso sin conexión.
3. Permite interfaces amigables y dinámicas sin esperar la respuesta del servidor.
4. Carga contenido de forma asíncrona sin recargar toda la página.
5. Comprueba la compatibilidad con el navegador y ayuda a resolver problemas de CSS.
6. Accede y modifica el DOM, la representación estructural de la página.

Alternativas: TypeScript, ELM, Dart y CoffeeScript, útiles en escenarios muy concretos.

Vídeo recomendado: «La historia completa de JavaScript» — EDteam, 24 min

Actividad 1 · 90 minutos

Caso práctico: definición inicial de la aplicación web HotelBook

Modalidad	Duración	Producto	Unidad
Equipos de 3-4	90 minutos	Informe breve + exposición	UD1

Reto Actuáis como equipo de desarrollo. Debéis transformar una necesidad de negocio en una propuesta técnica inicial coherente, justificando método, arquitectura y herramientas.

1.3 Herramientas del lado cliente

Para desarrollar necesitamos tres piezas básicas

Navegador

Sin él no pueden verse las aplicaciones en su contexto. Conviene trabajar con dos o tres navegadores y probar distintos tamaños de pantalla.

Editor de código

Validación de sintaxis, autocompletado, fragmentos, resaltado por colores, navegación entre archivos y extensiones.

Intérprete de JavaScript

Viene integrado en los navegadores: no requiere instalación adicional, solo habilitarlo en la configuración.

1.3.1 El navegador

Es el marco de ejecución de las aplicaciones web del lado cliente

Sin navegador no pueden verse las aplicaciones en su contexto real.

Motor propio

Cada navegador ejecuta el código con su motor JavaScript.

Chrome

Motor V8, en C++. Edge, Brave y Opera también lo usan.

Firefox

Motor SpiderMonkey, propio e independiente de Chromium.

Safari

Motor JavaScriptCore, de Apple, en macOS y iOS.

Cuota de mercado

Optimizar para todos no compensa: se optimiza para el más usado (Chrome, 69,23 % en agosto de 2025).

Siempre actualizado

Usar las últimas versiones evita problemas de seguridad y de incompatibilidad.

1.3.1 Probar y depurar en el navegador

Las herramientas de desarrollo (DevTools) acompañan todo el desarrollo

Consola

Errores y avisos, y ejecución de JavaScript sobre la página cargada.

Inspector de elementos

Explora el DOM y el CSS aplicado y prueba cambios en caliente.

Depurador

Puntos de interrupción, paso a paso e inspección de variables.

Monitor de red

Peticiones, tiempos de respuesta y tamaño de los recursos.

Rendimiento

Analiza la carga y el consumo para localizar cuellos de botella.

Almacenamiento

Revisa cookies, localStorage y datos de sesión.

Buenas prácticas: trabaja con dos o tres navegadores, porque lo que funciona en uno puede fallar en otro, y prueba distintos tamaños de pantalla con el modo responsive sin necesidad del dispositivo real.

1.3.2 El editor de código

Herramientas que acortan el tiempo de desarrollo y hacen el trabajo más eficiente

Permite abrir múltiples archivos a la vez e incorpora funciones pensadas para el desarrollador.

Validación de sintaxis

Comprueba errores conforme se escribe: un primer nivel de corrección.

Autocompletado

Sugerencias que agilizan la escritura y acortan los tiempos.

Fragmentos de código

Porciones frecuentes ya configuradas que evitan reescribirlas.

Resaltado con colores

Identifica cada elemento del programa de un simple vistazo.

Navegación de archivos

Mantiene localizados todos los ficheros y recursos del proyecto.

Complementos

Funcionalidad extendida según las tecnologías de cada proyecto.

Editor frente a IDE: el IDE es un conjunto de herramientas integradas —depuradores, analizadores de código, optimizadores, compiladores y control de versiones— en un mismo entorno.

1.3.2 Cómo elegir un editor de código

Criterios que conviene valorar antes de decidirse

Funciones básicas
Resaltado, sangría y finalización automáticas.

Experiencia de usuario
Menos fricción al escribir, más código final.

Facilidad de aprendizaje
Descartar los editores poco intuitivos.

Extensibilidad
Ampliar sus capacidades con herramientas adicionales.

Velocidad
No todos responden bien con poco hardware.

Sistema operativo
Comprobar si hay versión para Windows, Linux y macOS.

Compatibilidad con Git
Control de versiones, nativo o por extensión.

Soporte de la comunidad
Ayuda técnica rápida de la comunidad.

Lenguajes admitidos
Un proyecto suele mezclar varios lenguajes.

Precio
Gratuitos, con extras premium o de pago.

Atajos de teclado
Escribir más rápido sin usar el ratón.

Vista previa y errores
Ver el resultado sin cambiar de ventana.

1.3.2 Editores, IDE y extensiones

Qué se usa en la práctica

Editores de código

Visual Studio Code, Sublime Text, Atom, Brackets, Coda.

Entornos IDE

Visual Studio, Eclipse, NetBeans, WebStorm.

Editores online

CodePen, JSFiddle, JS Bin: probar código sin instalar nada en local.

Extensiones esenciales de Visual Studio Code

Prettier

Formatea los archivos automáticamente, al guardar o a demanda.

ESLint

Detecta errores de estilo y de programación que JavaScript no señala.

Live Server

Prueba la web en un servidor local de forma rápida y sencilla.

Lighthouse

Resalta fragmentos de código y ayuda a auditar calidad y rendimiento.

La calidad del código y el rendimiento de la aplicación dependen tanto del editor como de cómo lo configures.

1.3.3 El intérprete de JavaScript

La tercera pieza del kit de trabajo, y la que no hay que instalar

Es el motor que lee el código JavaScript y lo ejecuta línea a línea, sin compilarlo previamente.

Ya viene integrado

Todos los navegadores lo incorporan: no hay que instalar nada, solo habilitarlo.

Interpretado

Se ejecuta tal cual se escribe, sin compilar: los errores aparecen al ejecutar.

Fuera del navegador

Node.js usa el motor V8 para ejecutar JavaScript en el servidor.

Si el intérprete está deshabilitado en el navegador, la página se muestra pero pierde toda su parte dinámica: validaciones, menús, carga de contenido y cualquier interacción escrita en JavaScript.

Del código al resultado: escribes en el editor › el navegador carga la página › el intérprete ejecuta el script › ves el resultado

1.3.3 Probar JavaScript en la consola

El propio navegador es el banco de pruebas del intérprete

Abrir la consola

F12 o Ctrl+Mayús+I y pestaña Consola.

Escribir y ejecutar

Cada instrucción se evalúa al pulsar Intro.

Ver los errores

El mensaje indica archivo y número de línea.

Consola · DevTools

```
> let alumnos = 28;
> alumnos * 2
  56
> console.log("Hola, 2º DAW");
  Hola, 2º DAW
> alumno.nombre
  Uncaught ReferenceError: alumno is not defined
```

La consola permite probar instrucciones al vuelo y localizar los errores sin tocar el código fuente.

1.3.4 Otras herramientas útiles

Node.js

Plataforma sobre el motor V8 para aplicaciones escalables y en tiempo real. Convierte la máquina en servidor web y usa JavaScript en cliente y servidor.

Git y GitHub

Control de versiones local que registra los cambios y facilita la colaboración. GitHub centraliza los archivos y su historial para fusionar el trabajo del equipo.

Validadores

El validador del W3C comprueba que el HTML sigue el estándar; whatcms.org revela con qué tecnología está desarrollada una web.

1.3.4 Control de versiones

Cómo trabaja Git



Git organiza el trabajo en ramas paralelas que se fusionan en la rama principal hasta llegar a la versión estable.

1.3.4 Git y VS Code · Paso 1

Crear el repositorio del proyecto

1 **Crea tu cuenta en GitHub**

Entra en github.com y regístrate. Es gratuito y ahí vivirá la copia en la nube del proyecto.

2 **Pulsa el botón «New»**

En la pestaña Repositories, el botón verde «New» abre el formulario de un repositorio nuevo.

3 **Ponle nombre y visibilidad**

Escribe un nombre sin espacios (proyecto-web), elige Público o Privado y marca «Add a README file».

4 **Copia la dirección del repositorio**

Ya creado, el botón «Code» muestra la URL HTTPS. Cópiala: la necesitarás en el paso 2.



El formulario de creación del repositorio

Git y GitHub no son lo mismo: Git es el programa que guarda el historial en tu ordenador; GitHub es el servicio web donde se aloja esa copia para compartirla.

1.3.4 Git y VS Code · Paso 2

Enlazar el repositorio con tu carpeta de trabajo

1 **Abre VS Code y ve al panel de Git**

En la barra lateral izquierda, el icono «Control de código fuente» (tercero, con forma de ramas).

2 **Pulsa «Clonar repositorio»**

Pega la URL que copiaste en el paso 1 y pulsa Entrar. También sirve Ctrl+Mayús+P → «Git: Clone».

3 **Elige la carpeta del equipo**

VS Code descarga el proyecto dentro de esa carpeta y te ofrece abrirlo: di que sí.

4 **¿Ya tenías la carpeta creada?**

Ábrela y pulsa «Inicializar repositorio»; después «Publicar en GitHub» la enlaza con la nube.



Clonar el repositorio desde el editor

Qué significa clonar: descargar el repositorio y dejar la carpeta local conectada al remoto, de modo que el editor ya sabe a dónde enviar tus cambios.

1.3.4 Git y VS Code · Alternativa

Crear el repositorio directamente desde VS Code

1 · Conectar VS Code con tu cuenta de GitHub

1 — Inicia sesión en el editor

Pulsa el icono de la cuenta (abajo a la izquierda) → «Iniciar sesión con GitHub». Se abre el navegador y autorizas VS Code.

2 — Confirma la autorización

Al volver al editor, tu nombre de usuario aparece junto al icono: la conexión ya está hecha y no se repite.

3 — Comprueba la extensión

«GitHub Pull Requests» viene incluida; si no, instálala desde el panel de extensiones.

2 · Carpeta nueva → repositorio en GitHub

1 — Crea y abre la carpeta

Archivo → Abrir carpeta y elige (o crea) la carpeta vacía del proyecto, por ejemplo proyecto-web.

2 — Publicar en GitHub

En Control de código fuente pulsa «Publicar en GitHub»: el editor inicializa el repositorio local.

3 — Elige nombre y visibilidad

Escribe el nombre del repositorio y selecciona publicar como repositorio privado o público.

4 — Listo en la nube

Sube los archivos y ofrece «Abrir en GitHub»: el repositorio ya existe en tu cuenta.

No hace falta entrar en github.com: al publicar desde el editor se crea el repositorio remoto y se enlaza con tu carpeta local en un solo paso.

1.3.4 Git y VS Code · Paso 3

Mantener al día los cambios del proyecto

1 **Trabaja y guarda como siempre**

Al guardar un archivo, aparece en el panel de Git con una M (modificado) o una U (nuevo).

2 **Prepara los cambios con «+»**

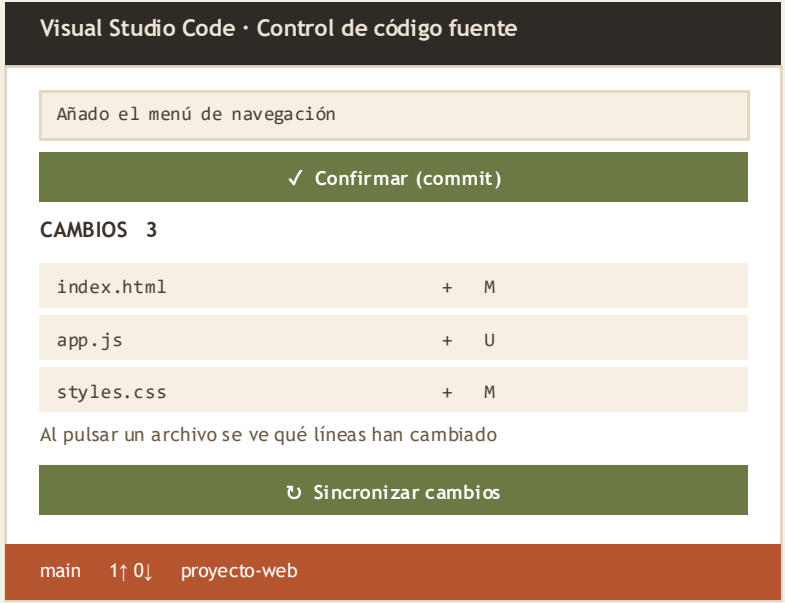
El «+» junto a cada archivo lo añade a la próxima confirmación. El «+» de arriba los añade todos.

3 **Escribe el mensaje y confirma**

Describe qué has hecho («Añado el menú de navegación») y pulsa Confirmar (Ctrl+Entrar).

4 **Sincroniza con GitHub**

«Sincronizar cambios» sube tus confirmaciones y baja las de tus compañeros. Repítelo a menudo.



El panel donde confirmas y sincronizas

Rutina diaria: guardar → preparar → confirmar con un mensaje claro → sincronizar. Cada confirmación es un punto al que siempre puedes volver.

1.4. Características de los lenguajes de script

Qué son, cómo funcionan y qué los diferencia

¿Qué es un lenguaje de script?

Un lenguaje interpretado que se ejecuta dentro de un programa anfitrión —el navegador, el servidor o el sistema operativo— para automatizar tareas y añadir comportamiento. No se compila: el intérprete lee y ejecuta el código línea a línea.

01 Interpretados

No se compilan: un intérprete (el motor del navegador) ejecuta el código directamente.

02 Embebidos en un anfitrión

Viven dentro de otro entorno —HTML, el navegador o Node.js— y usan sus recursos.

03 Tipado dinámico

Las variables no declaran tipo; se resuelve en tiempo de ejecución.

04 Sintaxis sencilla

Pocas reglas y escritura rápida: ideales para prototipar y hacer cambios ágiles.

05 Orientados a tareas

Automatizan acciones y gestionan la interacción del usuario con la página.

06 Multiplataforma

El mismo código funciona en cualquier sistema que disponga del intérprete.

Ejemplos: JavaScript en el navegador · PHP y Python en el servidor · Bash o PowerShell en el sistema.

1.5. Integración de JavaScript dentro de HTML

Tres formas de insertarlo

index.html

<head>

```
<script src="app.js" defer></script>
```

Carga un fichero .js externo

<body>

Contenido visible de la página

<script> … </script>

Código escrito dentro del propio HTML

1. Fichero externo (recomendado)

El JavaScript vive en su propio .js y el HTML lo enlaza.

```
<script src="app.js" defer></script>
```

2. Embebido en el HTML

El código se escribe entre etiquetas <script> dentro de la página.

```
<script> alert("Hola"); </script>
```

3. En línea en un atributo

Código dentro del propio elemento. Poco recomendable: mezcla estructura y lógica.

```
<button onclick="saluda()">Ir</button>
```

Cómo lo lee el navegador: recorre el HTML de arriba abajo y ejecuta cada <script> al encontrarlo; con defer espera a que la página esté construida.

1.5. Integración de JavaScript: dos ejemplos

EJEMPLO A · Fichero .js separado

Dos ficheros: la página y el código

index.html

```html
<!DOCTYPE html>
<html>
  <head>
    <script src="app.js" defer></script>
  </head>
  <body>
    <button id="btn">Saluda</button>
  </body>
</html>
```

app.js

```js
document.getElementById("btn")
  .addEventListener("click", () => {
    alert("¡Hola desde app.js!");
  });
```

EJEMPLO B · Código dentro del HTML

Un solo fichero: todo en la página

index.html

```html
<!DOCTYPE html>
<html>
  <body>
    <button id="btn">Saluda</button>

    <script>
      document.getElementById("btn")
        .addEventListener("click", () => {
          alert("¡Hola desde el HTML!");
        });
    </script>
  </body>
</html>
```

Los dos hacen lo mismo. El fichero separado permite reutilizar el código y mantenerlo mejor; el código embebido solo compensa en pruebas o scripts muy cortos.

1.5. Integración de JavaScript: dos ejemplos

EJEMPLO A · Fichero .js separado

Dos ficheros: la página y el código

index.html

```html
<!DOCTYPE html>
<html>
  <head>
    <script src="app.js" defer></script>
  </head>
  <body>
    <button id="btn">Saluda</button>
  </body>
</html>
```

app.js

```js
document.getElementById("btn")
  .addEventListener("click", () => {
    alert("¡Hola desde app.js!");
  });
```

EJEMPLO B · Código dentro del HTML

Un solo fichero: todo en la página

index.html

```html
<!DOCTYPE html>
<html>
  <body>
    <button id="btn">Saluda</button>

    <script>
      document.getElementById("btn")
        .addEventListener("click", () => {
          alert("¡Hola desde el HTML!");
        });
    </script>
  </body>
</html>
```

Los dos hacen lo mismo. El fichero separado permite reutilizar el código y mantenerlo mejor; el código embebido solo compensa en pruebas o scripts muy cortos.

1.7. Posibilidades que ofrece JavaScript

Qué puede cambiar en una página ya cargada

Tres ejemplos breves de lo que JavaScript puede hacer sobre una página ya cargada

1.7.1

Cambiar el contenido

Sustituir el texto de un párrafo al pulsar un botón, sin recargar la página ni volver al servidor.

```
elemento.innerHTML = "…"
```

1.7.2

Cambiar atributos

Modificar propiedades del elemento, como el src de una imagen, para que cambie con un clic.

```
imagen.src = "otra.png"
```

1.7.3

Cambiar el estilo

Retocar el CSS desde el código: color, tamaño o grosor del texto de forma inmediata.

```
texto.style.color = "crimson"
```

Todo ocurre en el navegador: JavaScript trabaja sobre el DOM, la representación en memoria de la página, y los cambios se ven al momento sin volver al servidor.

1.7.1. Modificar el contenido de la página

Cambiar el texto de un párrafo con un botón

```html
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Cambiar contenido HTML</title>
</head>
<body>
    <h1>Cambiar el contenido HTML</h1>
    <p id="mensaje">Este es el contenido original.</p>
    <button type="button" onclick="cambiarContenido()">Cambiar
contenido</button>


    <script>
        function cambiarContenido() {
            document.getElementById("mensaje").textContent =
                "Este contenido ha sido cambiado desde JavaScript.";
        }
    </script>
</body>
</html>
```



La clave: innerHTML sustituye el contenido del elemento que hemos localizado con getElementById.

1.7.2. Cambiar atributos de objetos HTML

Sustituir una imagen con un clic

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Cambiar atributos HTML</title>
</head>
<body>
    <h1>Cambiar atributos HTML</h1>
    <img id="imagen" src="https://picsum.photos/id/237/300/200" alt="Perro sentado">
    <p id="descripcion">Imagen original.</p>
    <button type="button" onclick="cambiarAtributos()">Cambiar imagen y atributos</button>


    <script>
        function cambiarAtributos() {
            const imagen = document.getElementById("imagen");
            const imagenInicial = "https://picsum.photos/id/237/300/200";
            const imagenAlternativa = "https://picsum.photos/id/1025/300/200";
            const mostrandoImagenInicial = imagen.getAttribute("src") === imagenInicial;


            if (mostrandoImagenInicial) {
```

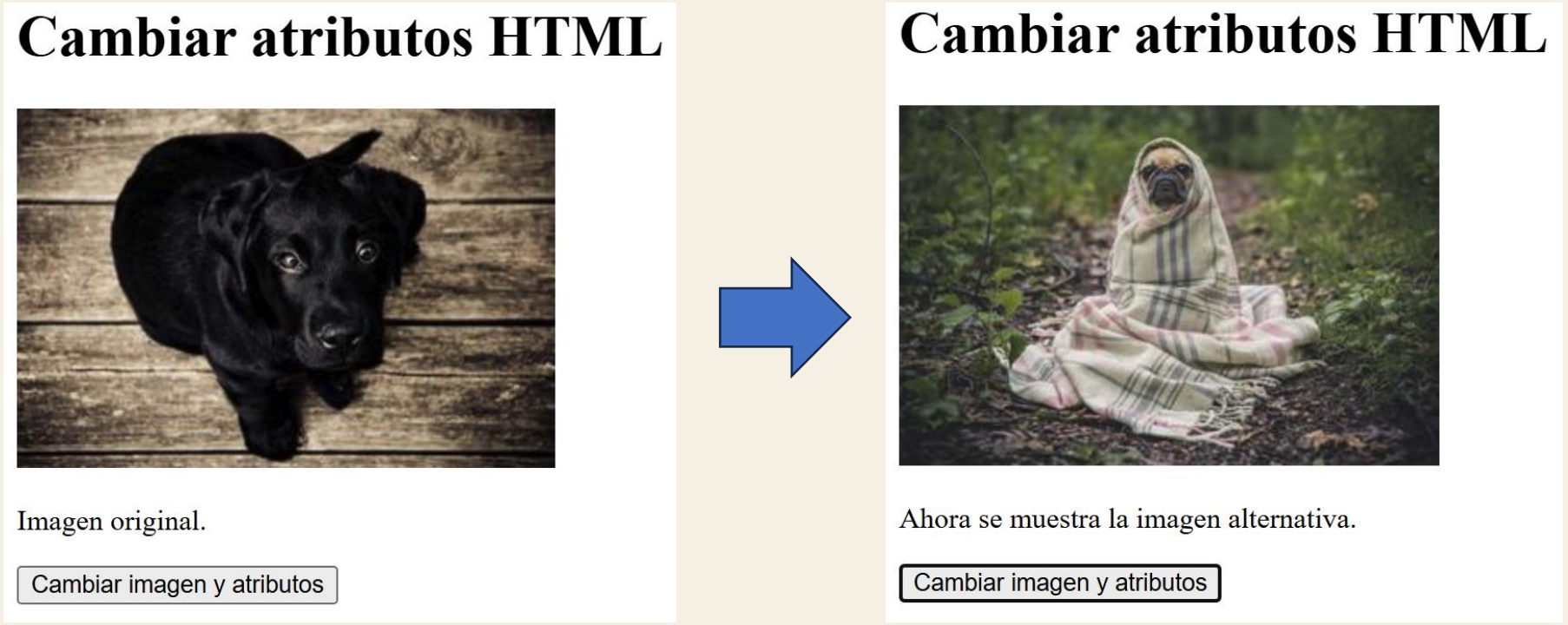
1.7.2. Cambiar atributos de objetos HTML

Sustituir una imagen con un clic

```
            if (mostrandoImagenInicial) {
                imagen.setAttribute("src", imagenAlternativa);
                imagen.setAttribute("alt", "Perro mirando a la camara");
                document.getElementById("descripcion").textContent =
                    "Ahora se muestra la imagen alternativa.";
            } else {
                imagen.setAttribute("src", imagenInicial);
                imagen.setAttribute("alt", "Perro sentado");
                document.getElementById("descripcion").textContent =
                    "Ahora se muestra la imagen inicial.";
            }
        }
    </script>
</body>
</html
```

1.7.2. Cambiar atributos de objetos HTML

Sustituir una imagen con un clic



1.7.3. Cambiar el estilo CSS

Retocar la apariencia del texto al pulsar un botón

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Cambiar estilos CSS</title>
    <style>
        #texto.estilo-alternativo {
            color: white;
            background-color: seagreen;
            font-size: 1.4rem;
            padding: 12px;
        }
    </style>
</head>
<body>
    <h1>Cambiar estilos CSS</h1>
    <p id="texto">Este parrafo tiene el estilo original.</p>
    <button type="button" onclick="cambiarEstilo()">Cambiar estilo</button>
```

1.7.3. Cambiar el estilo CSS

Retocar la apariencia del texto al pulsar un botón

```
    <script>
        function cambiarEstilo() {
            const texto = document.getElementById("texto");


            if (texto.classList.contains("estilo-alternativo")) {
                texto.classList.remove("estilo-alternativo");
                texto.textContent = "Este parrafo tiene el estilo original.";
            } else {
                texto.classList.add("estilo-alternativo");
                texto.textContent = "Este es el texto y el estilo alternativos.";
            }
        }
    </script>
</body>
</html>
```

La clave: la propiedad style da acceso al CSS de cada elemento; cambiarla afecta solo a ese elemento.

1.7.3. Cambiar el estilo CSS

Retocar la apariencia del texto al pulsar un botón



1.8. Comunicación de JavaScript con el exterior

Cómo mostrar mensajes desde nuestro código

Cuatro maneras de que un programa diga algo: a quien programa, a la página o directamente al usuario

EJEMPLO 1

Consola del navegador

Mensajes para quien programa. No los ve el usuario: aparecen en las herramientas de desarrollo (F12).

```
console.log("Hola")
```

EJEMPLO 2

Un párrafo del HTML

El mensaje se escribe dentro de un elemento de la página, en el sitio exacto que elijamos.

```
p.innerHTML = "Hola"
```

EJEMPLO 3

Escribir HTML directo

document.write vuelca contenido en el documento mientras se carga la página.

```
document.write("Hola")
```

EJEMPLO 4

Ventana de aviso

Un cuadro emergente que detiene la página hasta que el usuario pulsa Aceptar.

```
window.alert("Hola")
```

Elegir bien el canal: la consola sirve para depurar, el párrafo para informar dentro de la página, y la ventana de aviso solo cuando el mensaje no puede esperar.

1.8.1. Mensajes por consola y en un párrafo

console.log y innerHTML, uno para depurar y otro para el usuario

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Ejemplos de mensajes</title>
</head>
<body>
    <h1>Ejemplos de mensajes</h1>


    <h2>1. Mensaje en la consola</h2>
    <button type="button" onclick="mostrarEnConsola()">Mostrar mensaje en consola</button>


    <h2>2. Mensaje en un parrafo vacio</h2>
    <p id="mensaje"></p>
    <button type="button" onclick="mostrarEnParrafo()">Mostrar mensaje en el parrafo</button>
```

1.8.1. Mensajes por consola y en un párrafo

console.log y innerHTML, uno para depurar y otro para el usuario

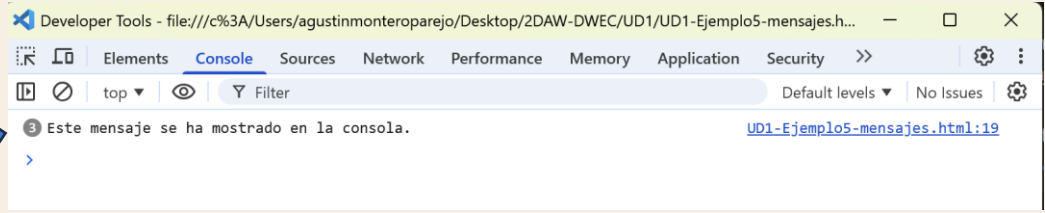
```
    <script>
        function mostrarEnConsola() {
            console.log("Este mensaje se ha mostrado en la consola.");
        }

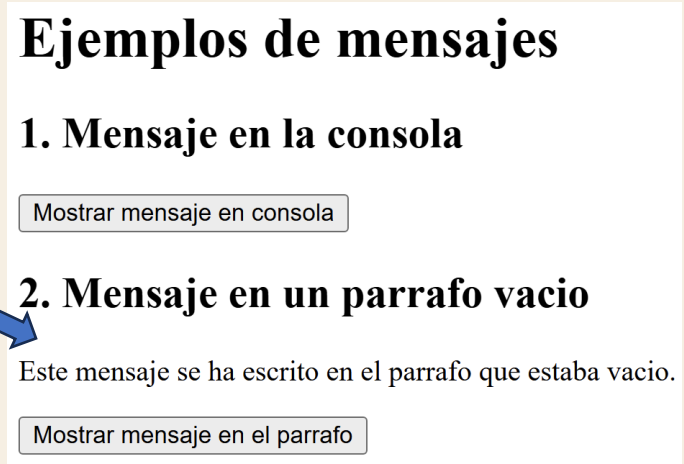

        function mostrarEnParrafo() {
            document.getElementById("mensaje").textContent =
                "Este mensaje se ha escrito en el parrafo que estaba vacio.";
        }
    </script>
</body>
</html>
```

1.8.1. Mensajes por consola y en un párrafo

console.log y innerHTML, uno para depurar y otro para el usuario







Abre la consola con F12

En la pestaña «Consola» del navegador se ven los mensajes de console.log y también los errores del código.

Diferencia clave: console.log no toca la página, solo deja un rastro para depurar; innerHTML sí modifica lo que el usuario ve.

1.8.2. Escribir HTML y avisar al usuario

document.write y window.alert

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Otros ejemplos de mensajes</title>
</head>
<body>
    <h1>Otros ejemplos de mensajes</h1>


    <h2>1. Escribir parrafos nuevos en HTML</h2>
    <button type="button" onclick="escribirParrafos()">Escribir parrafos</button>
    <div id="contenedor"></div>


    <h2>2. Mostrar un mensaje con alert</h2>
    <button type="button" onclick="mostrarAlerta()">Mostrar alert</button>
```

1.8.2. Escribir HTML y avisar al usuario

document.write y window.alert

```
    <script>
        function escribirParrafos() {
            const contenedor = document.getElementById("contenedor");
            const primerParrafo = document.createElement("p");
            const segundoParrafo = document.createElement("p");

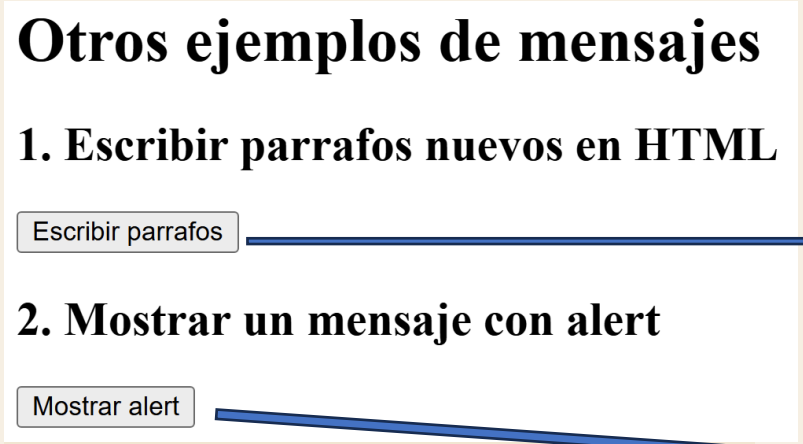
            primerParrafo.textContent = "Este es el primer parrafo escrito desde JavaScript.";
            segundoParrafo.textContent = "Este es el segundo parrafo escrito en el HTML.";

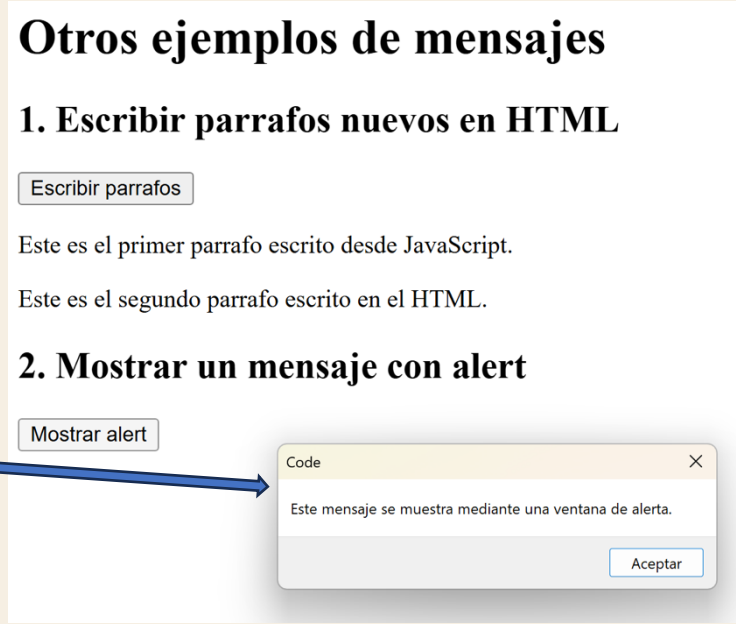
            contenedor.append(primerParrafo, segundoParrafo);
        }

        function mostrarAlerta() {
            alert("Este mensaje se muestra mediante una ventana de alerta.");
        }
    </script>
</body>
</html>
```

1.8.2. Escribir HTML y avisar al usuario

document.write y window.alert





Cuidado con document.write

Si se usa después de cargar la página, borra todo el contenido anterior. Hoy se prefiere innerHTML para escribir dentro de un elemento concreto.

window.alert bloquea la página: úsalo con moderación.

Cuando usar cada uno: document.write solo en ejemplos muy sencillos; window.alert para avisos que el usuario debe confirmar antes de seguir.

document.write construye la página; alert la interrumpe